Algoritmos y Estructuras de Datos II

Laboratorio - 21/05/2026

Laboratorio 5: TADs

`- Revisión 2026: Franco Luque / Marco Rocchietti`

Recursos generales:

- Videos del Laboratorio en el aula virtual
- Documentación en el aula virtual
  - **La biblia: El lenguaje de programación C.**
- Estilo de codificación:
  - Guía de estilo para la programación en C
  - Consejos de Estilo de Programación en C
- Más:
  - Referencia de C (cppreference)
  - The GNU C Library Reference Manual (en inglés)
  - Páginas del manual de linux: Comando “man 3 <*función*>”. Ejemplo: “man 3 scanf”.

Recursos específicos:

- Teóricos:
  - TADs
- Prácticos:
  - Práctico 2.2
  - Práctico 2.3

Ejercicio 1: TAD lista: Implementación con listas enlazadas

Dentro de la carpeta **`ej1`** se encuentran los siguientes archivos:

Archivo	*Descripción*
`list.h`	Especificación del TAD lista
`tests.c`	Tests para algunas funciones de consulta: `head`, `index` y `length`
`tests2.c`	Tests para más funciones: `tail`, `addr` y `take`

a) Leer y entender todo el código de testing.

b) Leer y entender la especificación del TAD lista dada en `list.h`.

Prestar atención las definiciones de los tipos `elem` y `list`:

```
typedef int elem;
typedef struct _list * list;
```

Observar que el **TAD no es polimórfico** (a diferencia del teórico), si no que los elementos son de tipo entero. Gracias al tipo `elem`, se facilita cambiar el tipo de los elementos, pero esto no lo hace polimórfico ya que una vez compilado, queda fijo el tipo de los elementos.

Observar también que el tipo `struct _list` no está definido. Este tipo depende de la implementación y por lo tanto se define en el siguiente ejercicio.

c) Implementar el TAD en un nuevo archivo `list.c` usando la idea de **listas enlazadas** como se vió en el lab04 (ejercicios 3, 4 y 5).
Para ello, **se debe definir el tipo `struct _list`** y se deben implementar cada uno de los constructores y operaciones declaradas en el archivo `list.h`:

- `empty`
- `addl`
- `is_empty`
- `head`
- `tail`
- `addr`
- `length`
- `concat`
- `index`
- `take`
- `drop`
- `copy_list`
- `destroy_list`

Compilar y testear con:

```
$ gcc -Wall -Wextra -pedantic -std=c99 list.c tests.c -o tests
$ ./tests
$ ./tests2
```

Asegurarse de que todos los tests pasan exitosamente.

Aclaración: Al compilar Ignorar el siguiente warning:

```
aviso: ISO C forbids empty initializer braces before C23 [-Wpedantic]
```

c) Agregar tests nuevos para todas las funciones testeadas (`head, index, length, tail, addr` y `take`). Por lo menos dos tests por función.

d) En `tests2.c` agregar nuevos tests para las funciones `drop` y `concat`. Como mínimo cuatro tests por función.

Ejercicio 2: Usando el TAD Lista

Dentro de la carpeta **`ej2`** se encuentran los siguientes archivos:

Archivo	*Descripción*
`main.c`	Contiene al programa principal que lee los números de un archivo para ser cargados en nuestra lista y obtener el promedio.
`array_helpers.h`	Contiene descripciones de funciones auxiliares para manipular arreglos.
`array_helpers.c`	Contiene implementaciones de dichas funciones.

a) Copiar los archivos `list.h` y `list.c` del ejercicio anterior.

b) Abrir el archivo **`main.c`** e implementar las funciones **`array_to_list()`** y **`average()`**. La función **`average()`** debe calcular el promedio de los elementos de la lista.

Una vez implementados los incisos **a)** y **b)**, compilar ejecutando:

```
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 -c list.c array_helpers.c main.c
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 list.o array_helpers.o main.o -o average
```

Ahora se puede ejecutar el programa corriendo:

```
$ ./average input/<file>.in
```

siendo **`<file>`** alguno de los nombres de archivo dentro de la carpeta **`input`**. Asegurar que el valor de los promedios que se imprimen en pantalla sean correctos y animense a definir sus propios casos de *input*.

Ejercicio 3: Listas con arreglos versión 1

a) Hacer una nueva implementación del TAD lista usando un arreglo y un número que indica el tamaño de la lista (como en los ejercicios práctico 2.2 ejercicio 2 y práctico 2.3 ejercicio 3a).

No modificar `list.h` de ninguna manera. Compilar y testear usando todos los tests realizados en el ejercicio 1.

Ayuda: El tipo se puede definir de la siguiente manera:

```
#define MAX_LENGTH 100

struct _list {
    elem elems[MAX_LENGTH];
    int size;
};
```

b) Resolver nuevamente el ejercicio 2 usando esta implementación en lugar de la de listas enlazadas. Para esto, sólo es necesario linkear usando la nueva versión de `list.o`. No se debe modificar ni recompilar `main.c`.

Ejercicio 4: Listas con arreglos versión 2

a) Hacer una nueva implementación del TAD lista usando la idea de **arreglos circulares** (como en el ejercicio práctico 2.3 ejercicio 3b). Las funciones `addr` y `addl` deben tener ambas **complejidad constante**.

En un **arreglo circular** se usa un índice `start` que indica la posición del arreglo donde comienzan los elementos de la lista y un número `size` que indica la cantidad de elementos. La lista puede dar "la vuelta" del arreglo, empezando al final del arreglo y terminando al principio.

Por ejemplo la lista: [8, 1, -2, 34, 4] se puede representar en un arreglo de 10 elementos de la siguiente manera:

indice:	0	1	2	3	4	5	6	7	8	9
valor:	34	4						8	1	-2

En este ejemplo, `start = 7` y `size = 5`.

Ayuda: Para las operaciones será necesario usar **aritmética modular**. Por ejemplo para la función index, la posición del arreglo a devolver es `(start + n) % MAX_ELEMS`.

b) Resolver nuevamente el ejercicio 2 usando esta implementación en lugar de la de listas enlazadas. Para esto, sólo es necesario linkear usando la nueva versión de `list.o`. No se debe modificar ni recompilar `main.c`.